Skrypt szkoleniowy — Dzień 1

Modularny monolit w ASP.NET Core Web API (.NET 10) — Car Sharing

Wersja: SQL Server 2022 + Autofac + MediatR + FluentValidation + EF Core

---

1. Cel dnia pierwszego

Celem pierwszego dnia jest zbudowanie od podstaw działającej aplikacji typu **ASP.NET Core Web API** w **C# / .NET 10**, która pokazuje podstawy **modularnego monolitu** na przykładzie systemu **car sharing**.

Na koniec dnia uczestnicy mają rozumieć i umieć samodzielnie odtworzyć:

- strukturę modularnego monolitu,
- podział odpowiedzialności na moduły,
- ideę publicznego API modułu,
- rejestrację modułów przez **Autofac**,
- automatyczną rejestrację klas `*Service` w obrębie modułu,
- komunikację synchroniczną między modułami,
- komunikację zdarzeniową in-process przez **MediatR**,
- walidację z użyciem **FluentValidation**,
- persystencję z użyciem **EF Core** i **SQL Server 2022**.

---

2. Założenia architektoniczne

2.1. Główne założenia

Budujemy **modularny monolit**, czyli:

- aplikacja działa jako jeden proces,
- wdrażamy ją jako jedną aplikację,
- ale wewnętrznie dzielimy ją na **moduły biznesowe**.

W naszym przypadku będą to trzy moduły:

- **Fleet** — zarządzanie flotą samochodów,
- **Reservations** — tworzenie rezerwacji,
- **Notifications** — reagowanie na zdarzenia biznesowe i zapisywanie logów powiadomień.

2.2. Żelazne reguły

1. Każdy moduł ma własną odpowiedzialność biznesową.
2. Każdy moduł jest właścicielem swoich danych.

3. Inny moduł nie sięga bezpośrednio do danych modułu obcego.
4. Każdy moduł wystawia swoje **publiczne API**.
5. Komunikacja odbywa się przez:
    - publiczne API modułu,
    - zdarzenia biznesowe.
6. Rejestracja zależności modułu jest zamknięta w klasie `NazwaModuluModule`.
7. Klasy niebędące encjami, kończące się na `Service`, rejestrujemy automatycznie przez skanowanie assembly modułu.

---

3. Stack technologiczny

W szkoleniu korzystamy z gotowych i popularnych bibliotek:

- **ASP.NET Core Web API**
- **Autofac**
- **MediatR**
- **FluentValidation**
- **Entity Framework Core**
- **Microsoft SQL Server 2022**

Nie piszemy ręcznie:

- własnego event brokera,
- własnego kontenera DI,
- własnego mechanizmu walidacji,
- własnej warstwy ORM.

---

4. Struktura solution

```
CarSharing
├── src
│   ├── CarSharing.Api
│   ├── CarSharing.BuildingBlocks
│   └── Modules
│       ├── CarSharing.Modules.Fleet
│       ├── CarSharing.Modules.Reservations
│       └── CarSharing.Modules.Notifications
└── tests
```

---

5. Tekst otwierający prowadzącego

„Dzisiaj nie budujemy zwykłego Web API. Budujemy aplikację, która ma zachowywać się jak modularny monolit. To oznacza, że od początku świadomie pilnujemy granic modułów, ich odpowiedzialności, komunikacji i sposobu rejestrowania zależności.

Nie będziemy ręcznie pisać rzeczy, które są już dobrze rozwiązane przez ekosystem .NET. Zamiast tego użyjemy Autofaca, MediatR, FluentValidation i EF Core. Dzięki temu skupimy się na architekturze, a nie na odtwarzaniu gotowych narzędzi.”

---

6. Tworzenie solution i projektów

6.1. Komendy

```
dotnet new sln -n CarSharing

mkdir src
mkdir tests

dotnet new webapi -n CarSharing.Api -o src/CarSharing.Api
dotnet new classlib -n CarSharing.BuildingBlocks -o src/CarSharing.BuildingBlocks

dotnet new classlib -n CarSharing.Modules.Fleet -o
src/Modules/CarSharing.Modules.Fleet
dotnet new classlib -n CarSharing.Modules.Reservations -o
src/Modules/CarSharing.Modules.Reservations
dotnet new classlib -n CarSharing.Modules.Notifications -o
src/Modules/CarSharing.Modules.Notifications

dotnet sln add src/CarSharing.Api
dotnet sln add src/CarSharing.BuildingBlocks
dotnet sln add src/Modules/CarSharing.Modules.Fleet
dotnet sln add src/Modules/CarSharing.Modules.Reservations
dotnet sln add src/Modules/CarSharing.Modules.Notifications
```

6.2. Referencje

```
dotnet add src/CarSharing.Api reference src/CarSharing.BuildingBlocks
dotnet add src/CarSharing.Api reference src/Modules/CarSharing.Modules.Fleet
dotnet add src/CarSharing.Api reference
src/Modules/CarSharing.Modules.Reservations
dotnet add src/CarSharing.Api reference
src/Modules/CarSharing.Modules.Notifications

dotnet add src/Modules/CarSharing.Modules.Fleet reference
src/CarSharing.BuildingBlocks
dotnet add src/Modules/CarSharing.Modules.Reservations reference
src/CarSharing.BuildingBlocks
```

```
dotnet add src/Modules/CarSharing.Modules.Notifications reference
src/CarSharing.BuildingBlocks
```

6.3. Tekst prowadzącego

„Host referuje moduły, ale moduły nie referują siebie nawzajem bez potrzeby. To jest zdrowy punkt startowy. Jeżeli moduły zaczną znać swoje wnętrza bezpośrednio, modularność bardzo szybko się rozpadnie.”

---

7. Instalacja pakietów

7.1. Pakiety

```
dotnet add src/CarSharing.Api package Autofac.Extensions.DependencyInjection
dotnet add src/CarSharing.Api package MediatR
dotnet add src/CarSharing.Api package
FluentValidation.DependencyInjectionExtensions

dotnet add src/Modules/CarSharing.Modules.Fleet package Autofac
dotnet add src/Modules/CarSharing.Modules.Fleet package MediatR
dotnet add src/Modules/CarSharing.Modules.Fleet package FluentValidation
dotnet add src/Modules/CarSharing.Modules.Fleet package
Microsoft.EntityFrameworkCore
dotnet add src/Modules/CarSharing.Modules.Fleet package
Microsoft.EntityFrameworkCore.SqlServer

dotnet add src/Modules/CarSharing.Modules.Reservations package Autofac
dotnet add src/Modules/CarSharing.Modules.Reservations package MediatR
dotnet add src/Modules/CarSharing.Modules.Reservations package FluentValidation
dotnet add src/Modules/CarSharing.Modules.Reservations package
Microsoft.EntityFrameworkCore
dotnet add src/Modules/CarSharing.Modules.Reservations package
Microsoft.EntityFrameworkCore.SqlServer

dotnet add src/Modules/CarSharing.Modules.Notifications package Autofac
dotnet add src/Modules/CarSharing.Modules.Notifications package MediatR
dotnet add src/Modules/CarSharing.Modules.Notifications package
Microsoft.EntityFrameworkCore
dotnet add src/Modules/CarSharing.Modules.Notifications package
Microsoft.EntityFrameworkCore.SqlServer
```

7.2. Tekst prowadzącego

„Autofac będzie odpowiadał za rejestrację zależności i modułów. MediatR wykorzystamy do komunikacji in-process. FluentValidation do walidacji komend. EF Core z providerem SQL Server do pracy z bazą.”

---

8. BuildingBlocks

W `CarSharing.BuildingBlocks` trzymamy tylko rzeczy naprawdę wspólne i techniczne.

8.1. ValidationBehavior

Plik: `ValidationBehavior.cs`

```
using FluentValidation;
using MediatR;

namespace CarSharing.BuildingBlocks;

public sealed class ValidationBehavior<TRequest, TResponse>(
    IEnumerable<IValidator<TRequest>> validators)
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull
{
    public async Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,
        CancellationToken cancellationToken)
    {
        if (!validators.Any())
        {
            return await next();
        }

        var context = new ValidationContext<TRequest>(request);

        var results = await Task.WhenAll(
            validators.Select(v => v.ValidateAsync(context, cancellationToken)));

        var failures = results
            .SelectMany(r => r.Errors)
            .Where(f => f is not null)
            .ToList();

        if (failures.Count != 0)
        {
            throw new ValidationException(failures);
        }

        return await next();
    }
}
```

8.2. Tekst prowadzącego

„To jest mały fragment infrastruktury aplikacyjnej. Nie budujemy własnego systemu walidacji. Łączymy po prostu MediatR i FluentValidation przez pipeline behavior.”

---

9. Host aplikacji i integracja z Autofacem

9.1. `Program.cs`

```
using Autofac;
using Autofac.Extensions.DependencyInjection;
using CarSharing.BuildingBlocks;
using CarSharing.Modules.Fleet;
using CarSharing.Modules.Fleet.Infrastructure;
using CarSharing.Modules.Notifications;
using CarSharing.Modules.Notifications.Infrastructure;
using CarSharing.Modules.Reservations;
using CarSharing.Modules.Reservations.Infrastructure;
using FluentValidation;
using MediatR;

var builder = WebApplication.CreateBuilder(args);

builder.Host.UseServiceProviderFactory(new AutofacServiceProviderFactory());

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblies(
        typeof(Program).Assembly,
        typeof(FleetModule).Assembly,
        typeof(ReservationsModule).Assembly,
        typeof(NotificationsModule).Assembly);
});

builder.Services.AddValidatorsFromAssemblyContaining<
    CarSharing.Modules.Fleet.Application.AddCar.AddCarCommandValidator>();

builder.Services.AddValidatorsFromAssemblyContaining<

CarSharing.Modules.Reservations.Application.CreateReservation.CreateReservationCom
mandValidator>();

builder.Services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(ValidationBehavior<,>));

builder.Host.ConfigureContainer<ContainerBuilder>(containerBuilder =>
{
    containerBuilder.RegisterModule(new FleetModule(builder.Configuration));
```

```
    containerBuilder.RegisterModule(new
ReservationsModule(builder.Configuration));
    containerBuilder.RegisterModule(new
NotificationsModule(builder.Configuration));
});

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();

using (var scope = app.Services.CreateScope())
{
    scope.ServiceProvider.GetRequiredService<FleetDbContext>
().Database.EnsureCreated();
    scope.ServiceProvider.GetRequiredService<ReservationsDbContext>
().Database.EnsureCreated();
    scope.ServiceProvider.GetRequiredService<NotificationsDbContext>
().Database.EnsureCreated();
}

app.Run();
```

9.2. Tekst prowadzącego

„Tutaj mamy dwa ważne elementy. Po pierwsze, ustawiamy Autofaca jako provider kontenera. Po drugie, w `ConfigureContainer` rejestrujemy moduły Autofaca. To jest dokładnie to miejsce, w którym host uruchamia moduły, ale nie zna ich wnętrza."

---

10. Konwencja modułu Autofaca

Każdy moduł wystawia klasę o nazwie:

- `FleetModule`
- `ReservationsModule`
- `NotificationsModule`

Ta klasa dziedziczy po `Autofac.Module` i odpowiada za:

- rejestrację `DbContext`,
- rejestrację publicznego API modułu,
- automatyczne skanowanie klas `*Service` w obrębie namespace modułu.

10.1. Reguła automatycznej rejestracji

Rejestrujemy automatycznie klasy:

- z assembly modułu,
- których nazwa kończy się na `Service`,
- które należą do przestrzeni nazw modułu.

Przykładowy wzorzec:

```
builder.RegisterAssemblyTypes(ThisAssembly)
    .Where(t =>
        t.Name.EndsWith("Service", StringComparison.Ordinal) &&
        t.Namespace is not null &&
        t.Namespace.StartsWith("CarSharing.Modules.Fleet",
StringComparison.Ordinal))
    .AsImplementedInterfaces()
    .InstancePerLifetimeScope();
```

10.2. Tekst prowadzącego

„Tę regułę wprowadzamy po to, żeby nie rejestrować ręcznie każdej klasy serwisowej. Ale jednocześnie chcemy, żeby skanowanie było ograniczone do modułu. Nie chcemy przypadkowo rejestrować wszystkiego z całego rozwiązania."

---

11. Moduł Fleet

11.1. Rola modułu

Fleet odpowiada za:

- samochody,
- ich podstawowe dane,
- dostępność samochodów.

11.2. Encja `Car`

```
namespace CarSharing.Modules.Fleet.Domain;

public sealed class Car
{
    public Guid Id { get; private set; }
    public string Brand { get; private set; } = default!;
    public string Model { get; private set; } = default!;
    public string RegistrationNumber { get; private set; } = default!;
    public bool IsAvailable { get; private set; }

    private Car() { }

    public Car(Guid id, string brand, string model, string registrationNumber)
    {
```

```
        Id = id;
        Brand = brand;
        Model = model;
        RegistrationNumber = registrationNumber;
        IsAvailable = true;
    }

    public void MarkUnavailable()
    {
        if (!IsAvailable)
        {
            throw new InvalidOperationException("Car is already unavailable.");
        }

        IsAvailable = false;
    }

    public void MarkAvailable()
    {
        if (IsAvailable)
        {
            throw new InvalidOperationException("Car is already available.");
        }

        IsAvailable = true;
    }
}
```

11.3. FleetDbContext

```
using CarSharing.Modules.Fleet.Domain;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Fleet.Infrastructure;

public sealed class FleetDbContext(DbContextOptions<FleetDbContext> options) :
DbContext(options)
{
    public DbSet<Car> Cars => Set<Car>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Car>(entity =>
        {
            entity.ToTable("fleet_cars");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.Brand).HasMaxLength(100).IsRequired();
            entity.Property(x => x.Model).HasMaxLength(100).IsRequired();
            entity.Property(x =>
x.RegistrationNumber).HasMaxLength(30).IsRequired();
```

```
            entity.HasIndex(x => x.RegistrationNumber).IsUnique();
            entity.Property(x => x.IsAvailable).IsRequired();
        });
    }
}
```

11.4. Publiczne API modułu

```
namespace CarSharing.Modules.Fleet.PublicApi;

public interface IFleetModuleApi
{
    Task<bool> IsCarAvailableAsync(Guid carId, CancellationToken cancellationToken
= default);
    Task MarkCarUnavailableAsync(Guid carId, CancellationToken cancellationToken =
default);
}
```

```
using CarSharing.Modules.Fleet.Infrastructure;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Fleet.PublicApi;

public sealed class FleetModuleApiService(FleetDbContext dbContext) :
IFleetModuleApi
{
    public async Task<bool> IsCarAvailableAsync(Guid carId, CancellationToken
cancellationToken = default)
    {
        var car = await dbContext.Cars.SingleOrDefaultAsync(x => x.Id == carId,
cancellationToken);
        return car is not null && car.IsAvailable;
    }

    public async Task MarkCarUnavailableAsync(Guid carId, CancellationToken
cancellationToken = default)
    {
        var car = await dbContext.Cars.SingleOrDefaultAsync(x => x.Id == carId,
cancellationToken);

        if (car is null)
        {
            throw new InvalidOperationException("Car not found.");
        }

        car.MarkUnavailable();
        await dbContext.SaveChangesAsync(cancellationToken);
```

```
    }
}
```

11.5. Komenda dodania auta

```
using MediatR;

namespace CarSharing.Modules.Fleet.Application.AddCar;

public sealed record AddCarCommand(
    string Brand,
    string Model,
    string RegistrationNumber) : IRequest<Guid>;
```

```
using FluentValidation;

namespace CarSharing.Modules.Fleet.Application.AddCar;

public sealed class AddCarCommandValidator : AbstractValidator<AddCarCommand>
{
    public AddCarCommandValidator()
    {
        RuleFor(x => x.Brand).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Model).NotEmpty().MaximumLength(100);
        RuleFor(x => x.RegistrationNumber).NotEmpty().MaximumLength(30);
    }
}
```

```
using CarSharing.Modules.Fleet.Domain;
using CarSharing.Modules.Fleet.Infrastructure;
using MediatR;

namespace CarSharing.Modules.Fleet.Application.AddCar;

public sealed class AddCarCommandHandler(FleetDbContext dbContext)
    : IRequestHandler<AddCarCommand, Guid>
{
    public async Task<Guid> Handle(AddCarCommand request, CancellationToken
cancellationToken)
    {
        var car = new Car(Guid.NewGuid(), request.Brand, request.Model,
request.RegistrationNumber);

        dbContext.Cars.Add(car);
        await dbContext.SaveChangesAsync(cancellationToken);

        return car.Id;
```

```
    }
}
```

11.6. Query pobrania floty

```
using MediatR;

namespace CarSharing.Modules.Fleet.Application.GetCars;

public sealed record GetCarsQuery() : IRequest<IReadOnlyList<CarDto>>;

public sealed record CarDto(
    Guid Id,
    string Brand,
    string Model,
    string RegistrationNumber,
    bool IsAvailable);
```

```
using CarSharing.Modules.Fleet.Infrastructure;
using MediatR;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Fleet.Application.GetCars;

public sealed class GetCarsQueryHandler(FleetDbContext dbContext)
    : IRequestHandler<GetCarsQuery, IReadOnlyList<CarDto>>
{
    public async Task<IReadOnlyList<CarDto>> Handle(GetCarsQuery request,
CancellationToken cancellationToken)
    {
        return await dbContext.Cars
            .AsNoTracking()
            .Select(x => new CarDto(
                x.Id,
                x.Brand,
                x.Model,
                x.RegistrationNumber,
                x.IsAvailable))
            .ToListAsync(cancellationToken);
    }
}
```

11.7. Kontroler Fleet

```
using CarSharing.Modules.Fleet.Application.AddCar;
using CarSharing.Modules.Fleet.Application.GetCars;
```

```
using MediatR;
using Microsoft.AspNetCore.Mvc;

namespace CarSharing.Modules.Fleet.Controllers;

[ApiController]
[Route("api/fleet")]
public sealed class FleetController(ISender sender) : ControllerBase
{
    [HttpPost("cars")]
    public async Task<IActionResult> AddCar([FromBody] AddCarCommand command,
CancellationToken cancellationToken)
    {
        var carId = await sender.Send(command, cancellationToken);
        return Ok(new { CarId = carId });
    }

    [HttpGet("cars")]
    public async Task<IActionResult> GetCars(CancellationToken cancellationToken)
    {
        var result = await sender.Send(new GetCarsQuery(), cancellationToken);
        return Ok(result);
    }
}
```

11.8. FleetModule

```
using Autofac;
using CarSharing.Modules.Fleet.Infrastructure;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;

namespace CarSharing.Modules.Fleet;

public sealed class FleetModule(IConfiguration configuration) : Module
{
    protected override void Load(ContainerBuilder builder)
    {
        builder.Register(ctx =>
            {
                var options = new DbContextOptionsBuilder<FleetDbContext>()
                    .UseSqlServer(configuration.GetConnectionString("FleetDb")!)
                    .Options;

                return new FleetDbContext(options);
            })
            .AsSelf()
            .InstancePerLifetimeScope();

        builder.RegisterAssemblyTypes(ThisAssembly)
            .Where(t =>
```

```
                t.Name.EndsWith("Service", StringComparison.Ordinal) &&
                t.Namespace is not null &&
                t.Namespace.StartsWith("CarSharing.Modules.Fleet",
StringComparison.Ordinal))
            .AsImplementedInterfaces()
            .InstancePerLifetimeScope();
    }
}
```

11.9. Tekst prowadzącego

„Zwróćcie uwagę, że cały moduł zamyka swoją rejestrację w jednej klasie Autofaca. To jest bardzo czytelne. Host nie zna wnętrza modułu. Moduł sam wie, co trzeba zarejestrować."

---

12. Moduł Reservations

12.1. Rola modułu

Reservations odpowiada za:

- tworzenie rezerwacji,
- sprawdzenie dostępności auta przez API modułu Fleet,
- zapis rezerwacji,
- publikację zdarzenia biznesowego.

12.2. Encja `Reservation`

```
namespace CarSharing.Modules.Reservations.Domain;

public sealed class Reservation
{
    public Guid Id { get; private set; }
    public Guid CarId { get; private set; }
    public Guid UserId { get; private set; }
    public DateTime FromUtc { get; private set; }
    public DateTime ToUtc { get; private set; }
    public string Status { get; private set; } = default!;

    private Reservation() { }

    public Reservation(Guid id, Guid carId, Guid userId, DateTime fromUtc,
DateTime toUtc)
    {
        Id = id;
        CarId = carId;
        UserId = userId;
        FromUtc = fromUtc;
        ToUtc = toUtc;
```

```
        Status = "Confirmed";
    }
}
```

12.3. ReservationsDbContext

```
using CarSharing.Modules.Reservations.Domain;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Reservations.Infrastructure;

public sealed class ReservationsDbContext(DbContextOptions<ReservationsDbContext>
options) : DbContext(options)
{
    public DbSet<Reservation> Reservations => Set<Reservation>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Reservation>(entity =>
        {
            entity.ToTable("reservations");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.CarId).IsRequired();
            entity.Property(x => x.UserId).IsRequired();
            entity.Property(x => x.FromUtc).IsRequired();
            entity.Property(x => x.ToUtc).IsRequired();
            entity.Property(x => x.Status).HasMaxLength(30).IsRequired();
        });
    }
}
```

12.4. Zdarzenie CarReserved

```
using MediatR;

namespace CarSharing.Modules.Reservations.Contracts;

public sealed record CarReserved(
    Guid ReservationId,
    Guid CarId,
    Guid UserId,
    DateTime FromUtc,
    DateTime ToUtc,
    DateTime OccurredOnUtc) : INotification;
```

12.5. Komenda utworzenia rezerwacji

```
using MediatR;

namespace CarSharing.Modules.Reservations.Application.CreateReservation;

public sealed record CreateReservationCommand(
    Guid CarId,
    Guid UserId,
    DateTime FromUtc,
    DateTime ToUtc) : IRequest<Guid>;
```

```
using FluentValidation;

namespace CarSharing.Modules.Reservations.Application.CreateReservation;

public sealed class CreateReservationCommandValidator :
AbstractValidator<CreateReservationCommand>
{
    public CreateReservationCommandValidator()
    {
        RuleFor(x => x.CarId).NotEmpty();
        RuleFor(x => x.UserId).NotEmpty();
        RuleFor(x => x.FromUtc).LessThan(x => x.ToUtc);
        RuleFor(x => x.ToUtc).GreaterThan(x => x.FromUtc);
    }
}
```

12.6. Service obsługujący rezerwację

```
using CarSharing.Modules.Fleet.PublicApi;
using CarSharing.Modules.Reservations.Contracts;
using CarSharing.Modules.Reservations.Domain;
using CarSharing.Modules.Reservations.Infrastructure;
using MediatR;

namespace CarSharing.Modules.Reservations.Application.CreateReservation;

public interface IReservationWorkflowService
{
    Task<Guid> CreateAsync(CreateReservationCommand request, CancellationToken
cancellationToken);
}

public sealed class ReservationWorkflowService(
    ReservationsDbContext dbContext,
    IFleetModuleApi fleetModuleApi,
    IPublisher publisher) : IReservationWorkflowService
{
    public async Task<Guid> CreateAsync(CreateReservationCommand request,
```

```
CancellationToken cancellationToken)
    {
        var isAvailable = await fleetModuleApi.IsCarAvailableAsync(request.CarId,
cancellationToken);

        if (!isAvailable)
        {
            throw new InvalidOperationException("Car is not available.");
        }

        await fleetModuleApi.MarkCarUnavailableAsync(request.CarId,
cancellationToken);

        var reservation = new Reservation(
            Guid.NewGuid(),
            request.CarId,
            request.UserId,
            request.FromUtc,
            request.ToUtc);

        dbContext.Reservations.Add(reservation);
        await dbContext.SaveChangesAsync(cancellationToken);

        await publisher.Publish(
            new CarReserved(
                reservation.Id,
                reservation.CarId,
                reservation.UserId,
                reservation.FromUtc,
                reservation.ToUtc,
                DateTime.UtcNow),
            cancellationToken);

        return reservation.Id;
    }
}
```

12.7. Handler komendy

```
using MediatR;

namespace CarSharing.Modules.Reservations.Application.CreateReservation;

public sealed class CreateReservationCommandHandler(
    IReservationWorkflowService reservationWorkflowService)
    : IRequestHandler<CreateReservationCommand, Guid>
{
    public Task<Guid> Handle(CreateReservationCommand request, CancellationToken
cancellationToken)
```

```
        => reservationWorkflowService.CreateAsync(request, cancellationToken);
}
```

12.8. Query rezerwacji

```
using MediatR;

namespace CarSharing.Modules.Reservations.Application.GetReservations;

public sealed record GetReservationsQuery() :
IRequest<IReadOnlyList<ReservationDto>>;

public sealed record ReservationDto(
    Guid Id,
    Guid CarId,
    Guid UserId,
    DateTime FromUtc,
    DateTime ToUtc,
    string Status);
```

```
using CarSharing.Modules.Reservations.Infrastructure;
using MediatR;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Reservations.Application.GetReservations;

public sealed class GetReservationsQueryHandler(ReservationsDbContext dbContext)
    : IRequestHandler<GetReservationsQuery, IReadOnlyList<ReservationDto>>
{
    public async Task<IReadOnlyList<ReservationDto>> Handle(GetReservationsQuery
request, CancellationToken cancellationToken)
    {
        return await dbContext.Reservations
            .AsNoTracking()
            .Select(x => new ReservationDto(
                x.Id,
                x.CarId,
                x.UserId,
                x.FromUtc,
                x.ToUtc,
                x.Status))
            .ToListAsync(cancellationToken);
    }
}
```

12.9. Kontroler

```
using CarSharing.Modules.Reservations.Application.CreateReservation;
using CarSharing.Modules.Reservations.Application.GetReservations;
using MediatR;
using Microsoft.AspNetCore.Mvc;

namespace CarSharing.Modules.Reservations.Controllers;

[ApiController]
[Route("api/reservations")]
public sealed class ReservationsController(ISender sender) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> CreateReservation(
        [FromBody] CreateReservationCommand command,
        CancellationToken cancellationToken)
    {
        var reservationId = await sender.Send(command, cancellationToken);
        return Ok(new { ReservationId = reservationId });
    }

    [HttpGet]
    public async Task<IActionResult> GetReservations(CancellationToken
cancellationToken)
    {
        var result = await sender.Send(new GetReservationsQuery(),
cancellationToken);
        return Ok(result);
    }
}
```

12.10. ReservationsModule

```
using Autofac;
using CarSharing.Modules.Reservations.Infrastructure;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;

namespace CarSharing.Modules.Reservations;

public sealed class ReservationsModule(IConfiguration configuration) : Module
{
    protected override void Load(ContainerBuilder builder)
    {
        builder.Register(ctx =>
            {
                var options = new DbContextOptionsBuilder<ReservationsDbContext>()

.UseSqlServer(configuration.GetConnectionString("ReservationsDb")!)
                    .Options;
```

```
                return new ReservationsDbContext(options);
            })
            .AsSelf()
            .InstancePerLifetimeScope();

        builder.RegisterAssemblyTypes(ThisAssembly)
            .Where(t =>
                t.Name.EndsWith("Service", StringComparison.Ordinal) &&
                t.Namespace is not null &&
                t.Namespace.StartsWith("CarSharing.Modules.Reservations",
StringComparison.Ordinal))
            .AsImplementedInterfaces()
            .InstancePerLifetimeScope();
    }
}
```

12.11. Tekst prowadzącego

„Tutaj bardzo wyraźnie widać sens publicznego API modułu. Reservations nie robi niczego bezpośrednio na danych Fleet. On pyta Fleet przez świadomie wystawiony kontrakt.”

---

13. Moduł Notifications

13.1. Rola modułu

Notifications:

- reaguje na zdarzenie `CarReserved`,
- zapisuje log powiadomienia,
- nie uczestniczy bezpośrednio w głównym przebiegu tworzenia rezerwacji.

13.2. Encja `NotificationLog`

```
namespace CarSharing.Modules.Notifications.Domain;

public sealed class NotificationLog
{
    public Guid Id { get; private set; }
    public string Message { get; private set; } = default!;
    public DateTime CreatedAtUtc { get; private set; }

    private NotificationLog() { }

    public NotificationLog(Guid id, string message, DateTime createdAtUtc)
    {
        Id = id;
        Message = message;
        CreatedAtUtc = createdAtUtc;
```

```
    }
}
```

13.3. NotificationsDbContext

```
using CarSharing.Modules.Notifications.Domain;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Notifications.Infrastructure;

public sealed class
NotificationsDbContext(DbContextOptions<NotificationsDbContext> options) :
DbContext(options)
{
    public DbSet<NotificationLog> NotificationLogs => Set<NotificationLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<NotificationLog>(entity =>
        {
            entity.ToTable("notification_logs");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.Message).HasMaxLength(500).IsRequired();
            entity.Property(x => x.CreatedAtUtc).IsRequired();
        });
    }
}
```

13.4. Handler zdarzenia

```
using CarSharing.Modules.Notifications.Domain;
using CarSharing.Modules.Notifications.Infrastructure;
using CarSharing.Modules.Reservations.Contracts;
using MediatR;

namespace CarSharing.Modules.Notifications.Application;

public sealed class CarReservedNotificationHandler(NotificationsDbContext
dbContext)
    : INotificationHandler<CarReserved>
{
    public async Task Handle(CarReserved notification, CancellationToken
cancellationToken)
    {
        var message =
            $"Reservation {notification.ReservationId} created for car
{notification.CarId} and user {notification.UserId}.";
```

```
        dbContext.NotificationLogs.Add(
            new NotificationLog(Guid.NewGuid(), message, DateTime.UtcNow));

        await dbContext.SaveChangesAsync(cancellationToken);
    }
}
```

13.5. Query logów

```
using MediatR;

namespace CarSharing.Modules.Notifications.Application.GetNotificationLogs;

public sealed record GetNotificationLogsQuery() :
IRequest<IReadOnlyList<NotificationLogDto>>;

public sealed record NotificationLogDto(
    Guid Id,
    string Message,
    DateTime CreatedAtUtc);
```

```
using CarSharing.Modules.Notifications.Infrastructure;
using MediatR;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Notifications.Application.GetNotificationLogs;

public sealed class GetNotificationLogsQueryHandler(NotificationsDbContext
dbContext)
    : IRequestHandler<GetNotificationLogsQuery, IReadOnlyList<NotificationLogDto>>
{
    public async Task<IReadOnlyList<NotificationLogDto>>
Handle(GetNotificationLogsQuery request, CancellationToken cancellationToken)
    {
        return await dbContext.NotificationLogs
            .AsNoTracking()
            .OrderByDescending(x => x.CreatedAtUtc)
            .Select(x => new NotificationLogDto(x.Id, x.Message, x.CreatedAtUtc))
            .ToListAsync(cancellationToken);
    }
}
```

13.6. Kontroler

```
using CarSharing.Modules.Notifications.Application.GetNotificationLogs;
using MediatR;
```

```
using Microsoft.AspNetCore.Mvc;

namespace CarSharing.Modules.Notifications.Controllers;

[ApiController]
[Route("api/notifications")]
public sealed class NotificationsController(ISender sender) : ControllerBase
{
    [HttpGet("logs")]
    public async Task<IActionResult> GetLogs(CancellationToken cancellationToken)
    {
        var result = await sender.Send(new GetNotificationLogsQuery(),
cancellationToken);
        return Ok(result);
    }
}
```

13.7. NotificationsModule

```
using Autofac;
using CarSharing.Modules.Notifications.Infrastructure;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;

namespace CarSharing.Modules.Notifications;

public sealed class NotificationsModule(IConfiguration configuration) : Module
{
    protected override void Load(ContainerBuilder builder)
    {
        builder.Register(ctx =>
            {
                var options = new DbContextOptionsBuilder<NotificationsDbContext>
()

.UseSqlServer(configuration.GetConnectionString("NotificationsDb")!)
                    .Options;

                return new NotificationsDbContext(options);
            })
            .AsSelf()
            .InstancePerLifetimeScope();

        builder.RegisterAssemblyTypes(ThisAssembly)
            .Where(t =>
                t.Name.EndsWith("Service", StringComparison.Ordinal) &&
                t.Namespace is not null &&
                t.Namespace.StartsWith("CarSharing.Modules.Notifications",
StringComparison.Ordinal))
            .AsImplementedInterfaces()
            .InstancePerLifetimeScope();
```

```
    }
}
```

---

14. appsettings.json

```
{
  "ConnectionStrings": {
    "FleetDb":
"Server=localhost;Database=CarSharingFleet;Trusted_Connection=True;TrustServerCert
ificate=True;",
    "ReservationsDb":
"Server=localhost;Database=CarSharingReservations;Trusted_Connection=True;TrustSer
verCertificate=True;",
    "NotificationsDb":
"Server=localhost;Database=CarSharingNotifications;Trusted_Connection=True;TrustSe
rverCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Tekst prowadzącego

„Tu rozdzielamy logicznie dane modułów. Nawet jeśli pracujemy na jednym serwerze SQL, każdy moduł ma własną bazę. To dobrze pokazuje własność danych. W praktyce można też użyć jednej fizycznej bazy i osobnych schematów, ale na szkoleniu ten wariant jest czytelniejszy.”

---

15. Scenariusz demonstracyjny

15.1. Dodanie samochodu

```
POST /api/fleet/cars
Content-Type: application/json

{
  "brand": "Toyota",
  "model": "Yaris",
  "registrationNumber": "WX12345"
}
```

15.2. Pobranie floty

```
GET /api/fleet/cars
```

15.3. Utworzenie rezerwacji

```
POST /api/reservations
Content-Type: application/json

{
  "carId": "TU_WKLEJ_ID_AUTA",
  "userId": "11111111-1111-1111-1111-111111111111",
  "fromUtc": "2026-04-20T08:00:00Z",
  "toUtc": "2026-04-20T12:00:00Z"
}
```

15.4. Pobranie rezerwacji

```
GET /api/reservations
```

15.5. Pobranie logów powiadomień

```
GET /api/notifications/logs
```

15.6. Tekst prowadzącego

„Teraz przechodzimy przez cały przepływ. Dodajemy auto, pobieramy flotę, tworzymy rezerwację i sprawdzamy, czy powstał log notyfikacji. Jeśli tak, to znaczy, że działa zarówno komunikacja synchroniczna przez publiczne API modułu, jak i komunikacja zdarzeniowa przez MediatR.”

---

16. Najważniejsze wnioski do omówienia

16.1. Modularny monolit to nie zwykły monolit z folderami

„Jeżeli każdy moduł ma tylko własny katalog, ale w praktyce wszystkie moduły sięgają do wszystkich danych, to nie mamy modularnego monolitu. Mamy zwykły monolit z ładniejszą strukturą."

16.2. Publiczne API modułu jest kluczowe

„Publiczne API modułu jest jednym z fundamentów. To przez nie inne moduły mają korzystać z funkcji modułu, a nie przez jego wnętrze."

16.3. MediatR nie zastępuje architektury

„MediatR pomaga zorganizować komunikację, ale sam w sobie nie rozwiązuje problemu granic modułów. Granice trzeba zaprojektować świadomie."

16.4. Autofac porządkuje rejestrację modułów

„Dzięki Autofacowi każdy moduł ma własne miejsce rejestracji. To jest dużo czytelniejsze niż trzymanie wszystkiego w jednym wielkim `Program.cs`."

---

17. Antywzorce

17.1. Bezpośredni dostęp do cudzego `DbContext`

Nie robimy tego:

```
public sealed class BadReservationHandler(FleetDbContext fleetDbContext)
{
}
```

17.2. Wspólne encje dla wszystkich modułów

Nie robimy wspólnej biblioteki z encjami typu:

- `Shared.Car`
- `Shared.Reservation`
- `Shared.User`

17.3. Jeden wspólny `Application` dla całego systemu

Nie budujemy jednego, centralnego projektu z całą logiką systemu.

17.4. Tekst prowadzącego

„Te antywzorce są bardzo kuszące, bo początkowo przyspieszają pracę. Problem w tym, że rozwalają granice modułów i potem każda zmiana zaczyna boleć cały system.”

---

18. Zadania dla kursantów

1. Dodaj endpoint oznaczający samochód jako dostępny.
2. Dodaj walidację, że rezerwacja nie może zaczynać się w przeszłości.
3. Dodaj drugi handler na zdarzenie `CarReserved`, który zapisze audyt.
4. Dodaj klasę `FleetAvailabilityService` i sprawdź, czy Autofac zarejestruje ją automatycznie.

Tekst prowadzącego

„To jest dobry moment, żeby samodzielnie sprawdzić, czy architektura jest przewidywalna. Jeżeli dobrze ją ustawiliśmy, nowe elementy powinny naturalnie trafiać w odpowiednie miejsce.”

---

19. Podsumowanie dnia 1

„Dzisiaj zbudowaliśmy zdrowy fundament modularnego monolitu. Mamy jeden host, ale logicznie odseparowane moduły. Mamy publiczne API modułu. Mamy komunikację przez MediatR. Mamy walidację przez FluentValidation. Mamy SQL Server i EF Core. I mamy Autofaca, który porządkuje rejestrację zależności w modelu modułowym.

To jest bardzo dobry punkt startowy przed dniem drugim, gdzie będziemy mogli dołożyć bardziej zaawansowane wzorce niezawodności i komunikacji.”

---

20. Tematy naturalnie przechodzące na dzień 2

- zdarzenia domenowe a integracyjne,
- asynchroniczność,
- outbox,
- inbox,
- idempotencja,
- korelacja wiadomości,
- Saga,
- droga od modularnego monolitu do mikroserwisów.